Tecnológico Nacional de México – Instituto Tecnológico de Tijuana
Ingeniería en Tecnologías de la Información y Comunicaciones

SEGUNDO AVANCE DEL PROYECTO FINAL

VIVA SOCCER

Sistema de gestión de alumnos para escuela de fútbol

Materia: Diseño y Evaluación de Interfaces de Usuario para Aplicaciones de Cómputo en la Nube

DOCENTE

Miguel Ángel López Ramírez

EQUIPO

José Luis Bustillos Ortiz – 22212153
Ramón Eduardo Contreras Barajas – 22211205
Alan Antonio Jiménez Salazar – 22211208

GRUPO

Aula: 9306
Paquete: 08B
Horario: 17:00–18:00



Tijuana, B.C., octubre de 2026

Dos flujos en paralelo, cinco fases

	01 Alineación inicial y arquitectura	02 Alta fidelidad y entorno	03 Design handoff y desarrollo	04 Pruebas y pre-despliegue	05 Producción y monitoreo
Flujo de Diseño (UX/UI)	Personas, user flows, arquitectura de información, wireframes	Design system y pantallas finales	Assets y especificación técnica	Design QA	Analítica de uso e iteración
Flujo Técnico (Ingeniería y DevOps)	Stack, nube y planeación de CDN	Repositorio y variables de entorno	Componentes reutilizables y CI	Staging, Lighthouse y E2E	CD a producción y monitoreo de errores

Cada fase separa los dos flujos. Lo que no existe en el repositorio se marca como Planeado o En proceso, con la herramienta propuesta.

VIVA SOCCER en una diapositiva

PROBLEMA

La evaluación ocurre en la cancha, pero se anota en libretas y hojas de cálculo; la versión anterior (C# WinForms) solo vivía en la computadora de la oficina.

SOLUCIÓN

- Aplicación web con una pantalla por rol
- Captura de pase, recepción y tiro pensada para el celular
- Histórico y progresión por alumno y por categoría
- Boletas imprimibles para entregar a los tutores

3 roles: administrador, secretaria y entrenador

7 pantallas y 3 formularios modales implementados

34 pruebas automatizadas en verde

Stack: lo propuesto contra lo que hay en el repositorio

Capa	Propuesta	En el repositorio	Estado
Backend	Django 5.2 · Python 3.12	Django 5.2.17 · Python 3.12.3	● Completo Igual
Interactividad	HTMX	HTMX 2.0.4 servido desde /static, sin CDN	● Completo Igual
Estilos	Tailwind CSS	Tailwind 3.4.17 con binario standalone: sin Node ni package.json	● En proceso Cambió la herramienta
Gráficas	Chart.js	Chart.js 4.4.1 local; paleta.js replica los tokens	● Completo Igual
Base de datos	PostgreSQL	SQLite en desarrollo; PostgreSQL aún sin configurar	● Pendiente Falta
Servidor	—	Gunicorn + WhiteNoise para producción	● Completo Agregado
Nube y CDN	Servicio con PostgreSQL y HTTPS	Sin proveedor configurado	● Planeado Falta

Por qué cambió: el binario de Tailwind evita un build de JavaScript, y HTMX y Chart.js se sirven locales porque en la cancha la señal es irregular.

Estado general por fase

Fase	Diseño (UX/UI)	Despliegue técnico	Global
1 · Alineación inicial y arquitectura	● En proceso Personas, casos de uso, mapa del sitio, flujo y heurísticas; faltan mapa de empatía y wireframes	● En proceso Stack definido; faltan nube y CDN	● En proceso
2 · Alta fidelidad y entorno	● Completo Design system y 7 pantallas en design.html	● En proceso GitHub y variables de entorno; faltan .env.example y DATABASE_URL	● Casi completa
3 · Design handoff y desarrollo	● Completo Tokens en tailwind.config.js y especificación en la sección 08	● En proceso Componentes reutilizables; falta el pipeline de CI	● En proceso
4 · Pruebas y pre-despliegue	● Planeado Design QA por realizar	● En proceso 34 pruebas; faltan staging, Lighthouse y E2E	● Pendiente
5 · Producción y monitoreo	● Planeado Analítica e iteración	● Planeado Hay base para producción; faltan CD y Sentry	● Pendiente

● Completo ● En proceso ● Planeado / Pendiente — *corte al 14 de octubre de 2026 (semana 4 del cronograma)*

01

Alineación inicial y arquitectura

UX/UI: personas, casos de uso, mapa del sitio y flujos.
Técnico: stack, proveedor de nube y planeación de CDN.

EN PROCESO

Fase 1 · Alineación inicial y arquitectura

FLUJO DE DISEÑO

UX/UI

COMPLETO	Tres proto-personas: coordinadora, entrenador y madre de familia (Propuesta §2.1)
COMPLETO	Seis casos de uso (CU-01 a CU-06) y once requerimientos priorizados con MoSCoW
COMPLETO	Mapa del sitio por rol y flujo de captura del entrenador (Figuras 1 y 2)
COMPLETO	Heurísticas de Nielsen traducidas a criterios propios de VIVA SOCCER
PLANEADO	Mapa de empatía en FigJam y wireframes de baja fidelidad en Figma (S4–S5)

FLUJO DE DESPLIEGUE TÉCNICO

Ingeniería y DevOps

COMPLETO	Frontend: plantillas Django + HTMX + Tailwind; sin SPA ni build de JavaScript
COMPLETO	Backend: Django 5.2 y Python 3.12, con roles por grupos de usuarios
PENDIENTE	Base de datos: PostgreSQL administrado (hoy solo SQLite)
PLANEADO	Nube: Render (web + PostgreSQL + HTTPS); Railway como alternativa
PLANEADO	CDN: Cloudflare frente al dominio; WhiteNoise ya comprime y cachea estáticos

Evidencia UX: tres personas, una por rol

Laura Méndez, 41

Coordinadora administrativa

Contexto: Escritorio, con interrupciones de padres que llegan a inscribir.

Necesita: Padrón al día, indicadores por categoría y teléfonos bien capturados.

"Necesito abrir algo y ver cómo vamos, no armar un Excel cada fin de mes."

Antonio "Toño" Ramírez, 27

Entrenador Sub-8 y Sub-10

Contexto: Cancha al aire libre, sol directo, prisa y una mano ocupada.

Necesita: Ver solo su grupo y calificar rápido con el pulgar, sin perder datos.

"Si me toma más de lo que dura un ejercicio, mejor lo apunto en la libreta."

Claudia Torres, 36

Madre de un alumno Sub-10

Contexto: Ratos cortos en el celular: transporte o descanso.

Necesita: Saber en qué mejora su hijo, sin lenguaje técnico.

"Me dicen que va bien, pero quisiera verlo con algo más que un 'échale ganas'."

Fuente: Propuesta de Proyecto Final, §2.1. Son proto-personas que se validan con entrevistas breves.

Evidencia UX: arquitectura de la información por rol

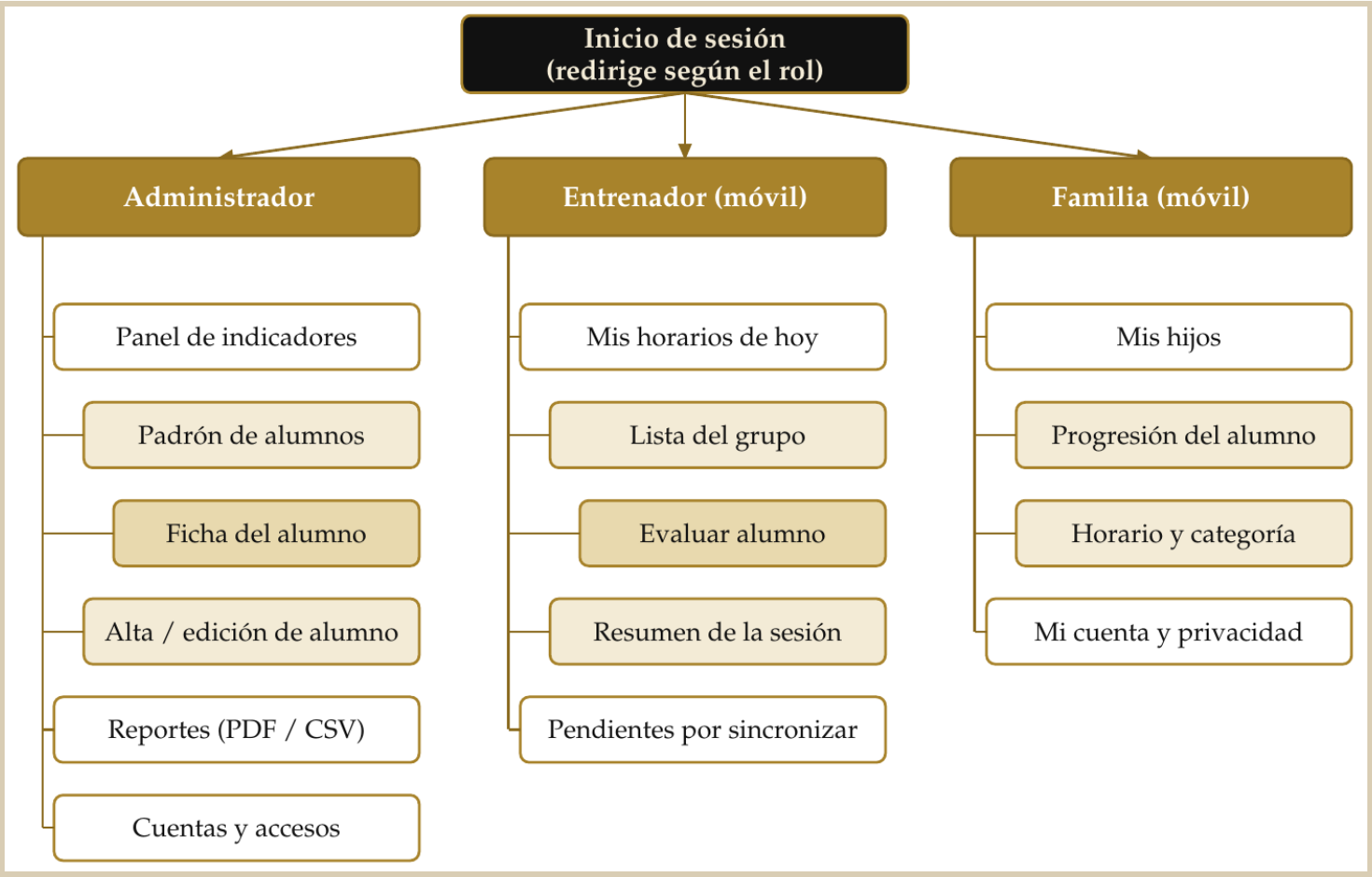


Figura 1 de la propuesta: mapa del sitio por rol.

LO QUE CAMBIÓ AL IMPLEMENTAR

- Cada rol entra directo a su pantalla, como se diseñó
- Nuevo rol Secretaria: padrón, tutores, corrección de calificaciones y boletas
- Tutor como registro propio: los hermanos comparten tutor
- Portal de Familia (RF-09) y cuentas desde la UI (RF-11): sin empezar

Evidencia UX: flujo de captura del entrenador

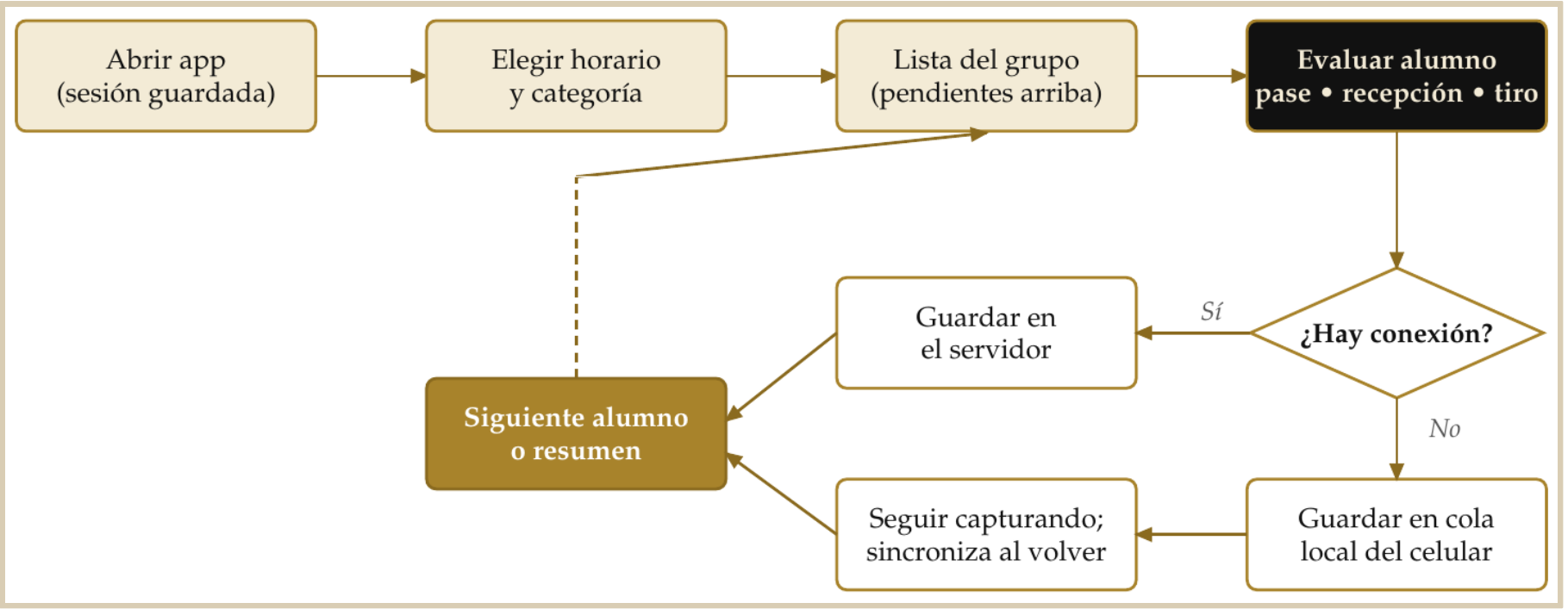


Figura 2 de la propuesta: flujo de captura con manejo de la conexión.

Implementado: elegir grupo y fecha, lista del grupo, guardado en bloque y una sola evaluación por alumno y fecha.

Pendiente: la rama sin conexión (RF-06), con un Service Worker y una cola en IndexedDB, sin agregar un build de JS.

Evidencia UX: heurísticas de Nielsen aplicadas

Heurística	Qué revisaremos (propuesta)	Qué hay hoy en la app
1. Visibilidad del estado	Confirmación al guardar; indicador de pendientes	Mensajes de guardado y barra de avance en captura
2. Mundo real	Vocabulario de la escuela	Categorías, horarios, pase, recepción y tiro
3. Control y libertad	Corregir sin salir de la lista	Reenviar la captura actualiza; la ficha permite editar
4. Consistencia	Mismos componentes en web y móvil	Tokens únicos en tailwind.config.js
5. Prevención de errores	Una evaluación por fecha; teléfono validado	Restricción en la base de datos y validación del formato
6. Reconocer antes que recordar	Folio e iniciales visibles	Folio VS-0000 en padrón, ficha y captura
7. Flexibilidad y eficiencia	Avance al siguiente pendiente	Pendiente: hoy se guarda el grupo en bloque
8. Estética minimalista	Solo lo necesario para calificar	Captura con tres campos por alumno
9. Recuperarse de errores	Mensajes claros	Errores por campo en español
10. Ayuda	Indicaciones en el portal de familias	Pendiente con el portal

Alcance: requerimientos de la propuesta contra el repositorio

ID	Requerimiento	Prioridad	Estado
RF-01	Inicio de sesión y redirección por rol	Must	● Completo
RF-02	Alta y edición de alumnos con tutor y folio	Must	● Completo
RF-03	Padrón con búsqueda y filtros	Must	● En proceso · Falta buscar por folio
RF-04	Captura de pase, recepción y tiro	Must	● Completo
RF-05	Una evaluación por alumno y fecha	Must	● Completo
RF-06	Captura sin conexión	Should	● Pendiente
RF-07	Indicadores del panel	Must	● En proceso · Faltan grupos sin evaluar
RF-08	Ficha con historial y gráfica	Must	● Completo
RF-09	Portal para familias	Should	● Pendiente
RF-10	Reportes PDF y CSV	Should	● En proceso · Boletas imprimibles; falta CSV
RF-11	Gestión de cuentas	Could	● En proceso · Solo desde el Django admin

6 de 8 requerimientos Must completos; los dos restantes, en proceso.

02

Diseño de alta fidelidad y configuración del entorno

UX/UI: design system y pantallas finales en alta fidelidad.
Técnico: repositorio en GitHub y variables de entorno.

CASI COMPLETA

Fase 2 · Alta fidelidad y configuración del entorno

FLUJO DE DISEÑO

UX/UI

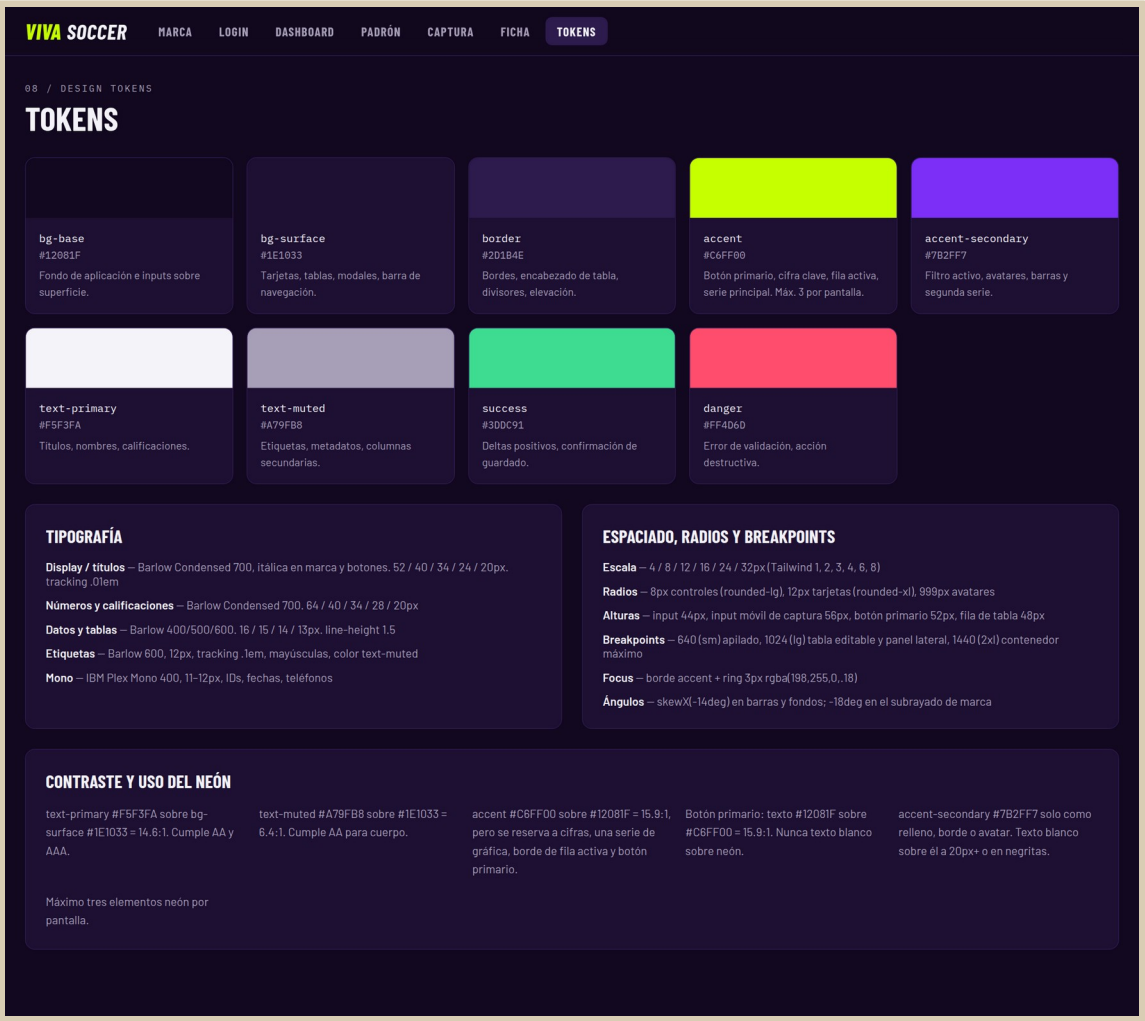
COMPLETO	Design system en design/design.html: 9 colores, 3 familias tipográficas, espaciado y radios
COMPLETO	Componentes: botón primario y fantasma, campos, chips, tarjetas y campo de calificación
COMPLETO	Alta fidelidad: marca, login, panel, padrón, formulario, captura y ficha
EN PROCESO	Tutores y boleta se implementaron sin maqueta; falta llevarlas a design.html

FLUJO DE DESPLIEGUE TÉCNICO

Ingeniería y DevOps

COMPLETO	Repositorio en GitHub: joseluis-buo/viva_soccer, rama main, 7 commits
COMPLETO	Variables de entorno DJANGO_*: clave secreta, DEBUG, hosts y orígenes CSRF
COMPLETO	.env fuera de git; con DEBUG apagado la app exige la clave secreta
PLANEADO	.env.example y DATABASE_URL (dj-database-url + psycopg) para PostgreSQL
PLANEADO	URL de pruebas: entorno de staging en Render

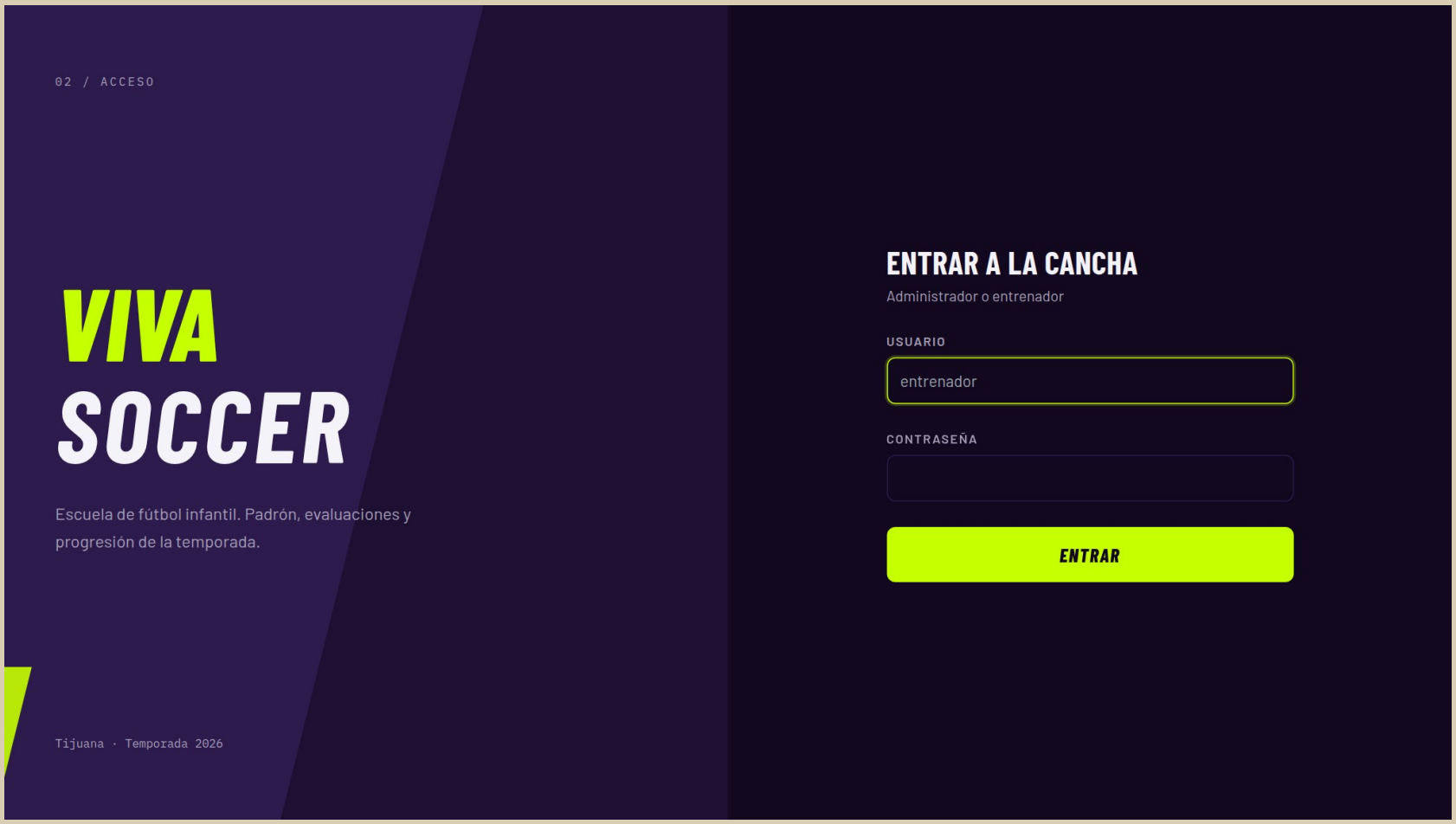
Design system: los tokens de design.html



DECISIONES

- Fondo oscuro y neón para leer al sol: texto principal a 14.6:1 y secundario a 6.4:1 (WCAG AA)
- El neón se reserva para la acción principal y las cifras clave: máximo tres por pantalla
- Barlow Condensed para títulos y calificaciones, Barlow para datos e IBM Plex Mono para folios y fechas
- Alturas táctiles: campos de 44 px, calificación en captura de 56 px y botón primario de 52 px

Pantalla final: inicio de sesión



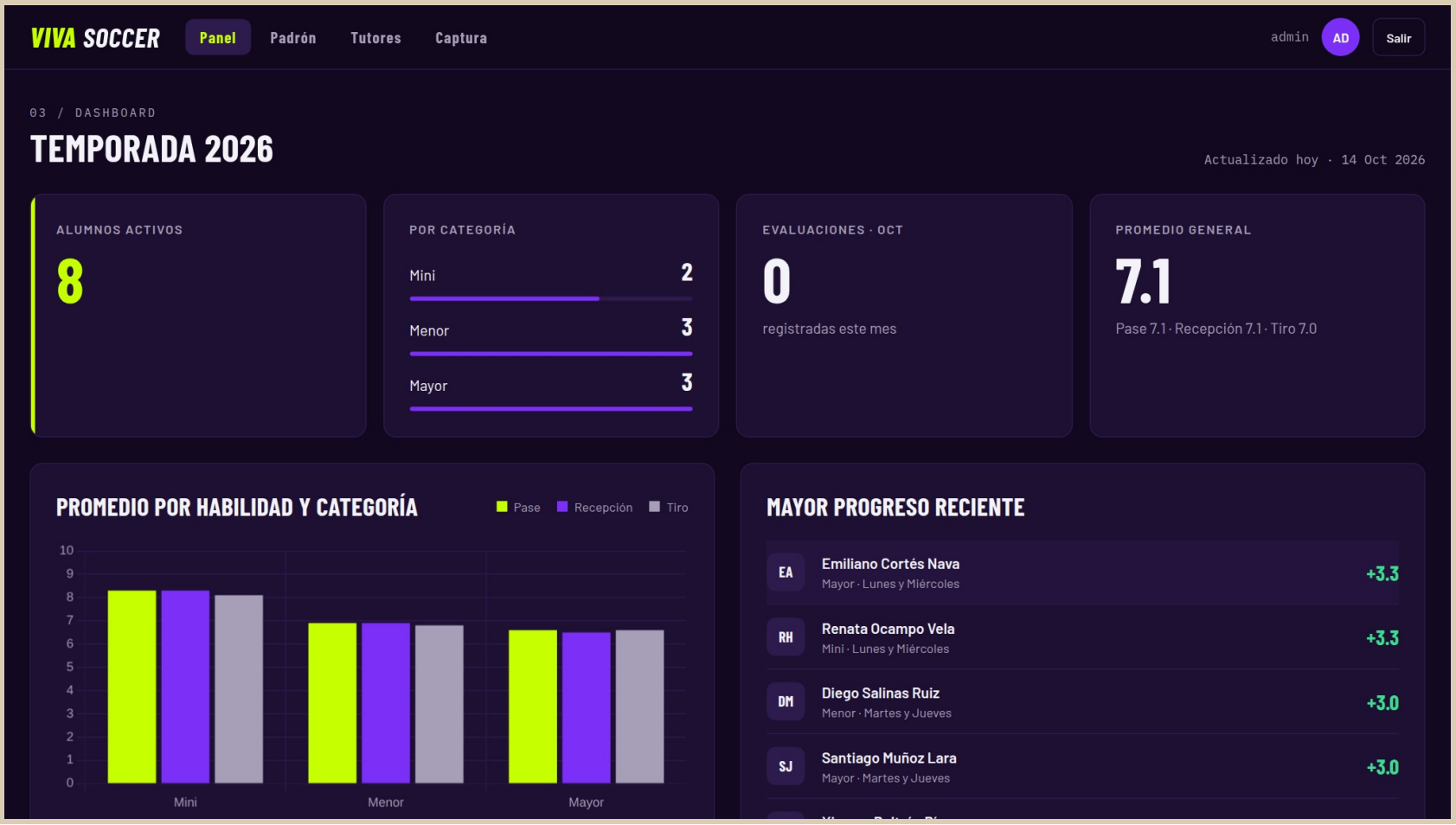
ROL

Todos

DECISIÓN DE DISEÑO

Un solo formulario para todos; tras entrar, cada rol cae directo en su pantalla (RF-01). El neón solo aparece en el botón ENTRAR.

Pantalla final: panel del administrador



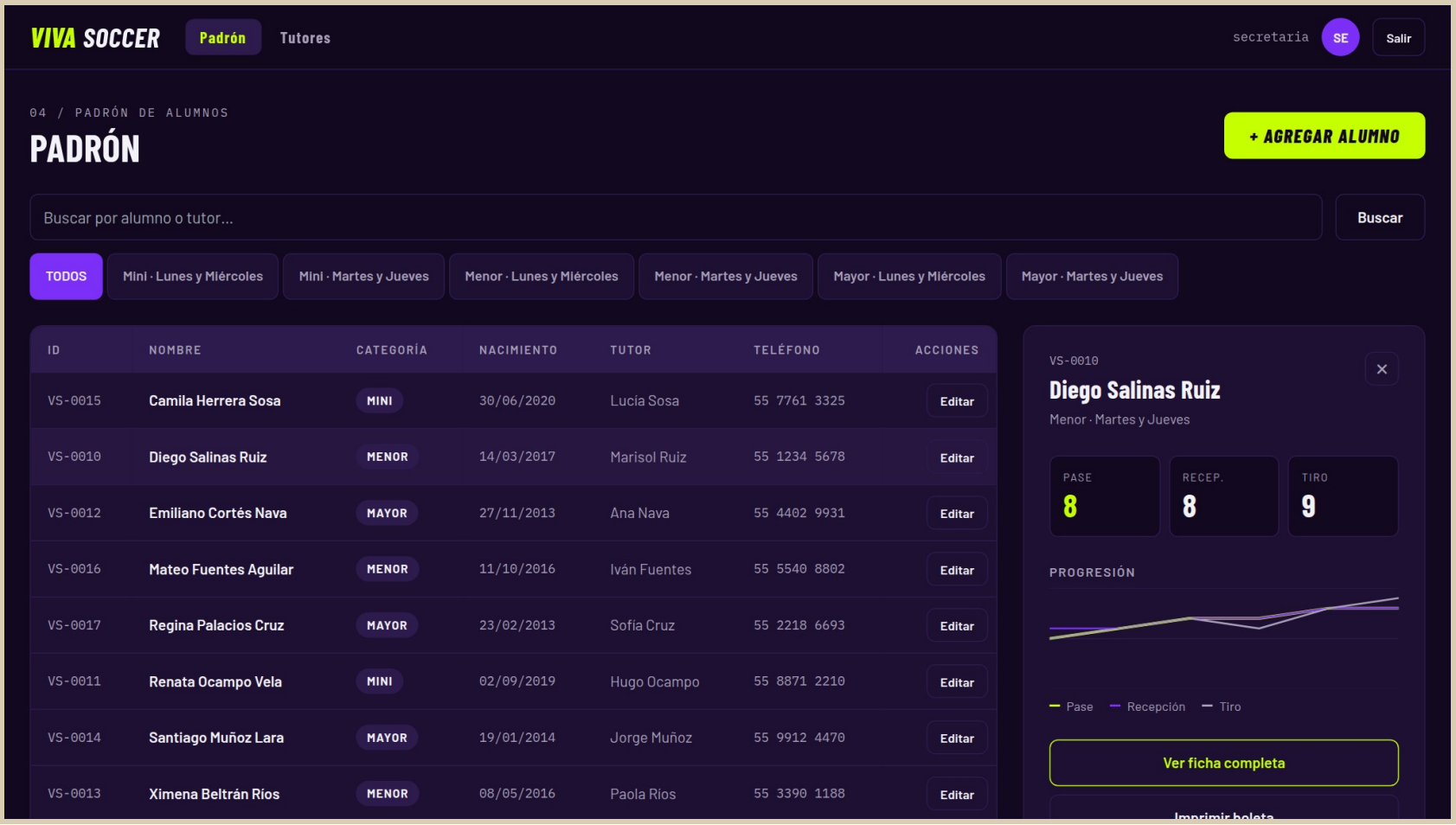
ROL

Administrador

DECISIÓN DE DISEÑO

Las cuatro cifras de la temporada arriba y, debajo, el promedio por habilidad y categoría. La gráfica usa los mismos tokens que la interfaz.

Pantalla final: padrón de alumnos



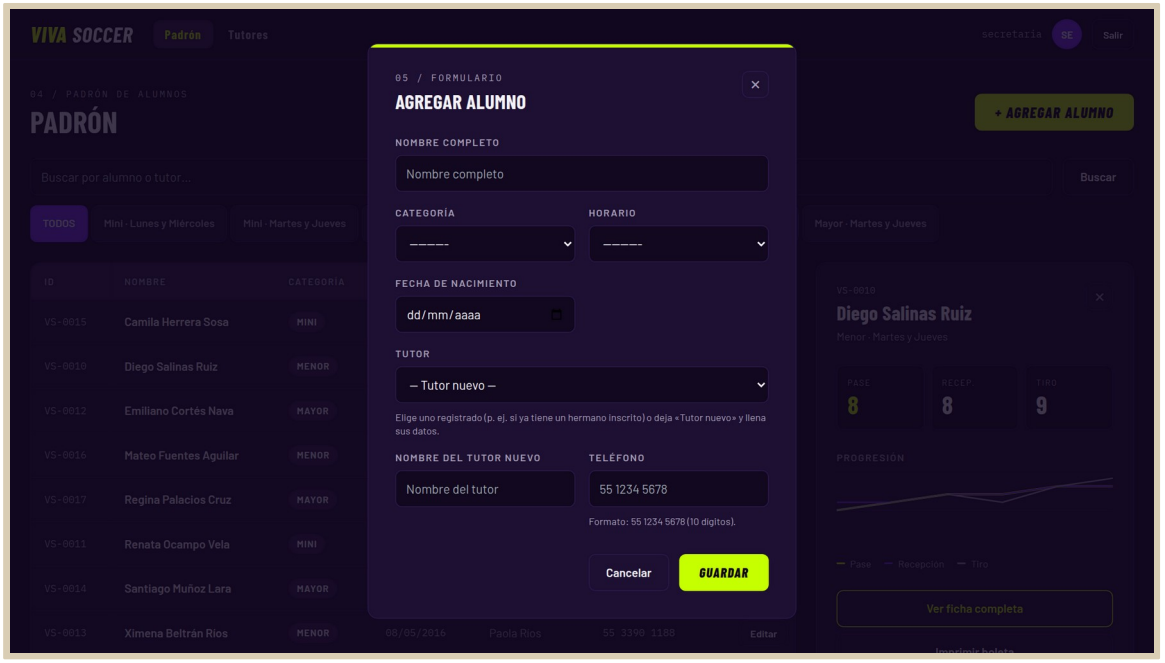
ROL

Secretaria y administrador

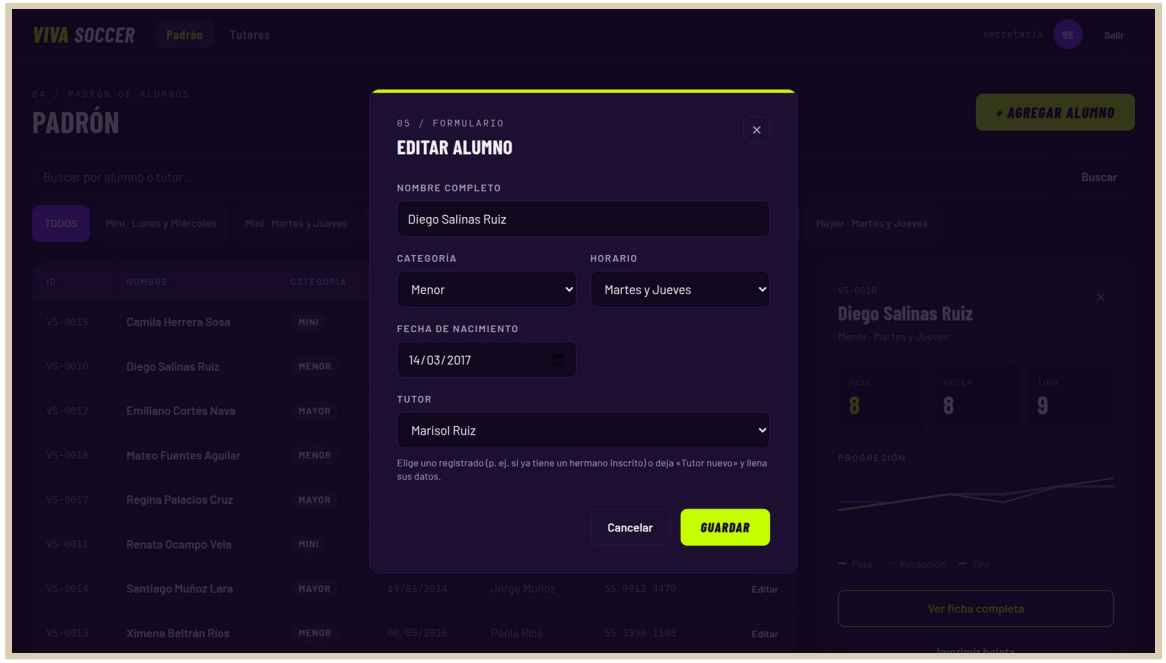
DECISIÓN DE DISEÑO

La tabla y el detalle conviven: un clic en la fila carga el panel lateral con HTMX sin salir de la lista. Los chips filtran por grupo.

Pantalla final: alta y edición de alumno



Alta: si el tutor es nuevo, se captura en el mismo formulario.

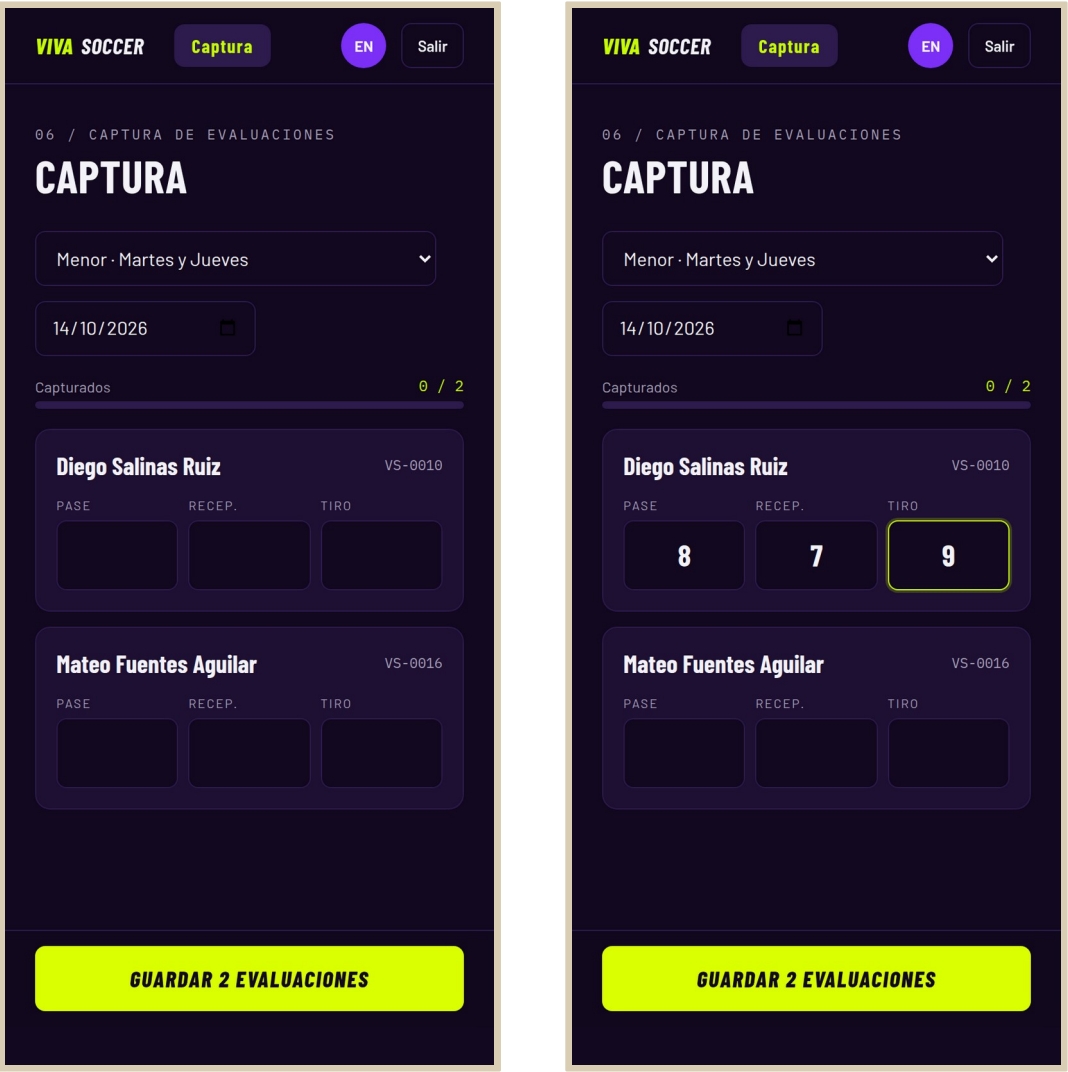


Edición: se elige un tutor ya registrado.

DECISIÓN DE DISEÑO

Un modal sobre el padrón conserva el contexto. Un hermano reutiliza al tutor ya registrado, y el teléfono se valida con el formato 55 1234 5678.

Pantalla final: captura de calificaciones en el celular



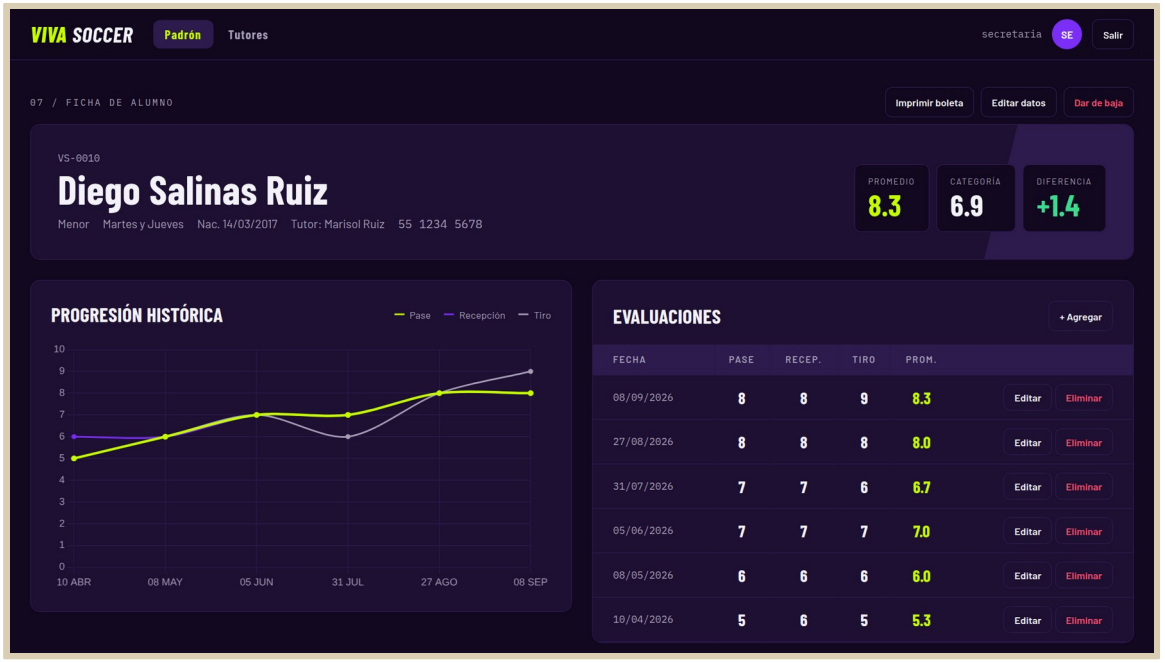
ROL

Entrenador (mobile-first, 390 px)

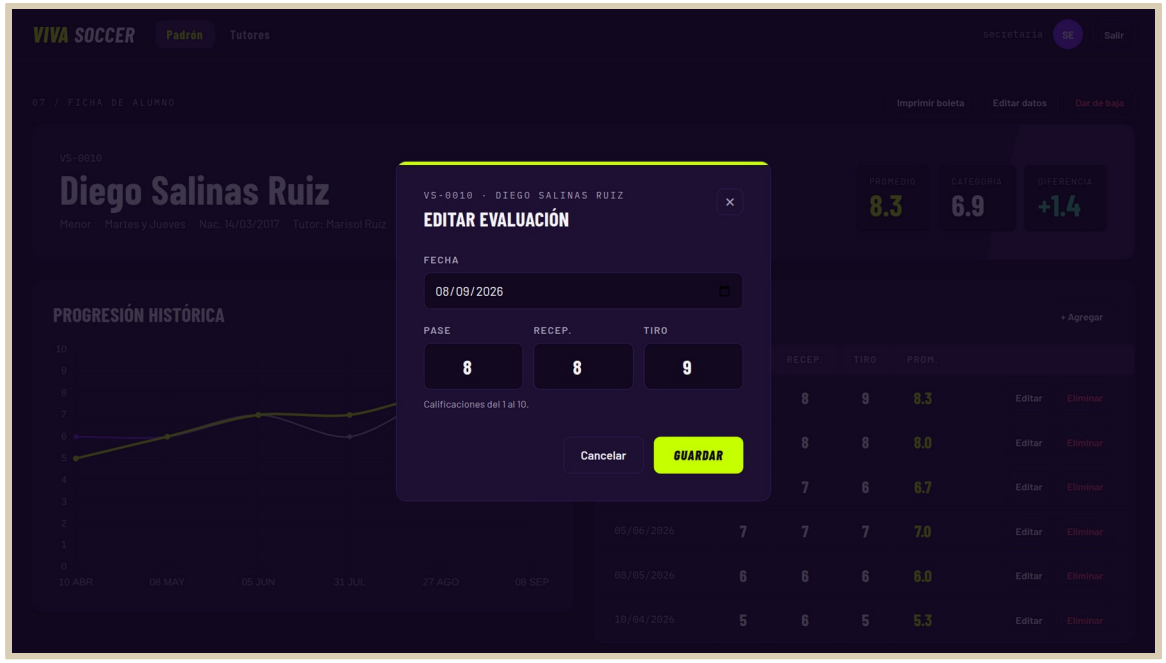
DECISIÓN DE DISEÑO

- Una tarjeta por alumno con pase, recepción y tiro: sin navegar entre pantallas
- Campos numéricos de 56 px: más que el mínimo de 44 px para el pulgar
- Botón de guardado fijo abajo y barra de avance del grupo
- Reenviar la captura actualiza la evaluación en lugar de duplicarla

Rol secretaria: ficha del alumno y corrección de calificaciones



Ficha: imprimir boleta, editar, dar de baja y editar evaluaciones.

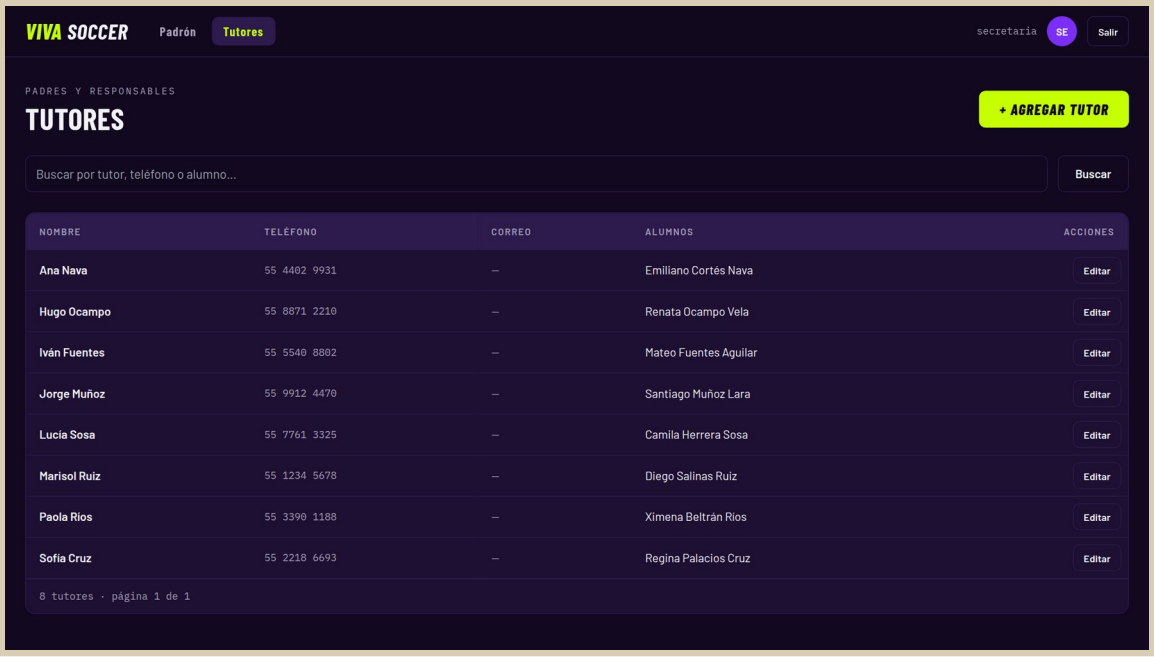


Corrección de una evaluación en un modal.

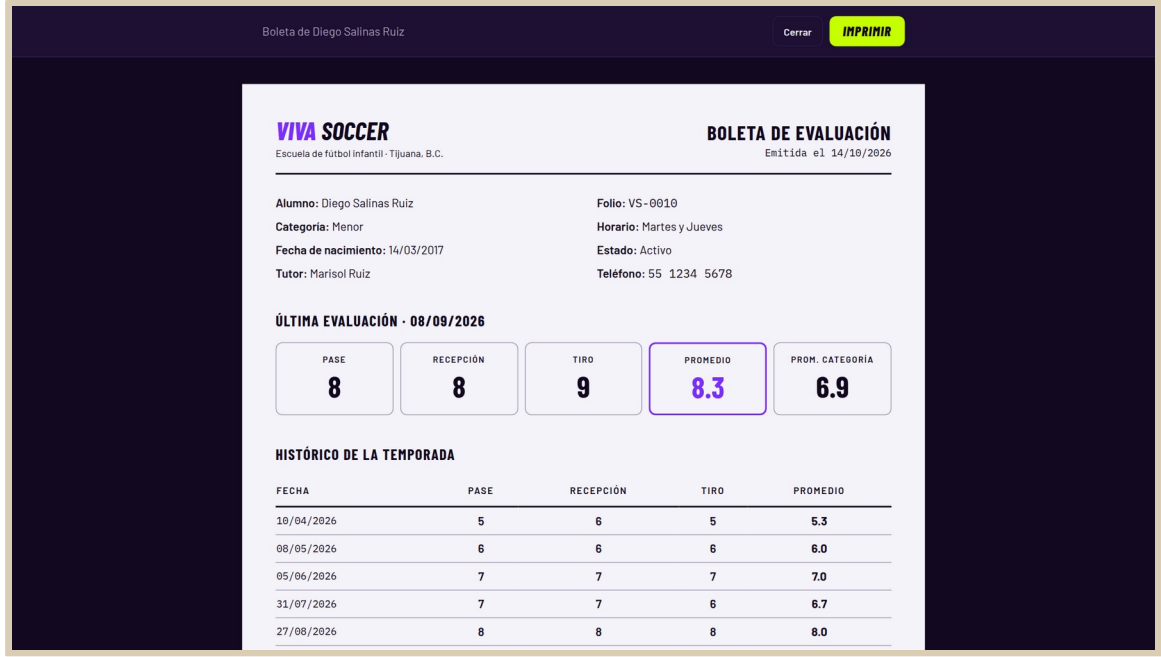
DECISIÓN DE DISEÑO

Las acciones de oficina viven en la ficha, donde la secretaria ya tiene el contexto. La baja conserva el histórico y no permite dos evaluaciones en la misma fecha.

Rol secretaria: tutores y boletas



Tutores: alta, edición y búsqueda por tutor, teléfono o alumno.



Boleta: hoja carta lista para imprimir desde el navegador.

DECISIÓN DE DISEÑO

La boleta invierte la paleta (tinta oscura sobre papel) para imprimir sin gastar tóner. Se imprime por alumno o por grupo completo, una hoja por niño.

03

Design handoff y desarrollo

UX/UI: assets exportados y especificación técnica.
Técnico: componentes reutilizables y pipeline de CI.

EN PROCESO

Fase 3 · Design handoff y desarrollo

FLUJO DE DISEÑO

UX/UI

COMPLETO	Tokens de design.html llevados a tailwind.config.js con paleta cerrada
COMPLETO	paleta.js replica los tokens para Chart.js: las gráficas no usan colores sueltos
COMPLETO	Especificación en la sección 08: tipografías, tamaños, alturas, breakpoints y focus
COMPLETO	Assets: el logo es tipográfico y las fuentes vienen de Google Fonts; no hay imágenes que exportar

FLUJO DE DESPLIEGUE TÉCNICO

Ingeniería y DevOps

COMPLETO	Clases de componente: .btn-primario, .btn-fantasma, .campo, .chip, .tarjeta, .nota-input
COMPLETO	Parciales y modales HTMX reutilizables (alumno, tutor y evaluación)
COMPLETO	Permisos reutilizables por rol: solo_administrador, solo_secretaria y solo_entrenador
PLANEADO	CI en GitHub Actions: check, migraciones, pruebas y build de Tailwind en cada push

Handoff: del token al componente y a la pantalla

1 · TOKEN

tailwind.config.js

```
'accent': '#C6FF00',
// Botón primario, cifra clave,
// fila activa, serie principal
```

2 · COMPONENTE

static/src/input.css

```
.btn-primario {
  @apply h-[52px] rounded-lg
    bg-accent text-bg-base
    font-display italic;
}
```

3 · PANTALLA

templates/escuela/*.html

```
<button class="btn-primario">
  GUARDAR
</button>
```

- Regla del proyecto: ningún color hexadecimal en las plantillas; todos salen de los tokens.
- Cambiar un token actualiza botones, tablas y gráficas a la vez, y lo evaluado en el diseño es lo mismo que se implementa.

04

Pruebas y pre-despliegue

UX/UI: Design QA.
Técnico: staging con previews automáticos, Lighthouse y pruebas E2E.

PENDIENTE

Fase 4 · Pruebas y pre-despliegue

FLUJO DE DISEÑO

UX/UI

EN PROCESO	Contraste documentado en el design system: 14.6:1, 6.4:1 y 15.9:1 (WCAG AA)
PLANEADO	Design QA contra design.html a 360, 768 y 1280 px con capturas de Playwright
PLANEADO	Evaluación heurística 1 sobre wireframes (S5) y 2 sobre el prototipo (S8)
PLANEADO	Pruebas con 10 usuarios, pensamiento en voz alta y cuestionario SUS (S10)

FLUJO DE DESPLIEGUE TÉCNICO

Ingeniería y DevOps

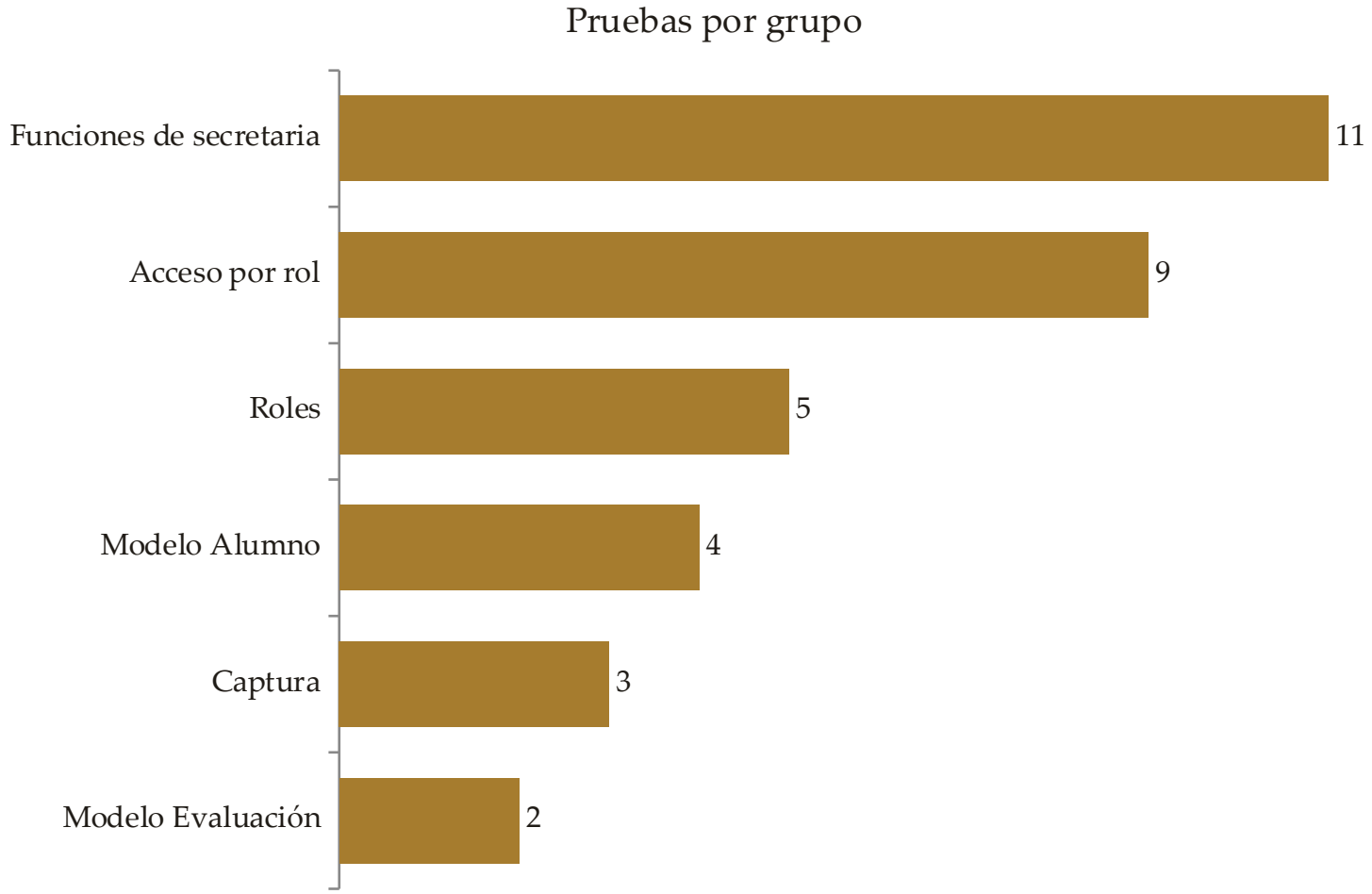
COMPLETO	34 pruebas automatizadas de dominio, permisos por rol, captura y secretaria
PLANEADO	Previews por pull request con Render Preview Environments
PLANEADO	Lighthouse CI: carga en menos de 3 s en 4G simulada (RNF-03)
PLANEADO	E2E con Playwright: el recorrido de estas capturas pasa a ser una suite en el repo

Lo que ya se prueba: 34 pruebas automatizadas

34

pruebas con python manage.py test, todas en verde

Aún no corren solas: el siguiente paso es ejecutarlas en GitHub Actions en cada push.



05

Producción y monitoreo

UX/UI: analítica de uso e iteración.
Técnico: CD a producción y monitoreo de errores.

PENDIENTE

Fase 5 · Producción y monitoreo

FLUJO DE DISEÑO

UX/UI

PLANEADO — Analítica de uso con Plausible: sin cookies, adecuada al manejar datos de menores

PLANEADO — Medir las metas de la propuesta: 20 s o menos por alumno, 90 % de éxito y SUS de 75 o más

PLANEADO — Iterar hasta no dejar problemas de severidad 3 o 4 sin resolver (OE7)

FLUJO DE DESPLIEGUE TÉCNICO

Ingeniería y DevOps

COMPLETO — Base para producción: Gunicorn, WhiteNoise y configuración por variables de entorno

PLANEADO — CD: GitHub Actions despliega en Render cuando main pasa las pruebas

PLANEADO — Monitoreo de errores con Sentry (sentry-sdk para Django)

PLANEADO — Health check de Render y alertas en horarios de entrenamiento (RNF-04)

Siguientes pasos

	S5 19 oct	S6 26 oct	S7–S8 2–9 nov	S9–S10 16–23 nov	S11–S12 30 nov–7 dic
Diseño (UX/UI)	Mapa de empatía, wireframes y heurística 1	Guía visual con las pantallas de secretaria	Prototipo en Figma y heurística 2	Pruebas con 10 usuarios y SUS	Iteración y reporte final
Técnico	PostgreSQL y .env.example	CI en GitHub Actions y staging en Render	Captura sin conexión (RF-06)	E2E con Playwright y Lighthouse CI	CD a producción y Sentry

Semanas del cronograma de la propuesta (Figura 3).

Conclusiones

01

Base funcional sólida

Los requerimientos Must centrales funcionan de punta a punta y están cubiertos por 34 pruebas.

02

Diseño y código alineados

El design system pasó al código sin pérdidas: los colores solo existen como tokens.

03

El rezago es de despliegue

PostgreSQL, CI/CD, staging y monitoreo concentran el trabajo técnico que falta.

04

Falta la voz del usuario

El mapa de empatía, los wireframes y las pruebas con usuarios cerrarán el flujo de UX.